Reviewer Pack — Tropical Geometric Invariant of Graphs vs. AC0 Parity Depth

Entry 48da56aad654 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 05:30:51 UTC

Tropical Geometric Invariant of Graphs vs. AC0 Parity Depth

Entry ID: `48da56aad654` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 05:30:51 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: AC0 Parity Depth

Statement:

> ['For any graph G with n vertices, the tropical Euler characteristic χ_t(G) of its associated matroid is lower-bounded by a constant c such that χ_t(G) ≥ c·log(n)', 'where χ_t(G) is computed using the maximum entry-wise nonnegative real numbers in the tropicalization of the determinant of the incidence matrix of G.', 'Additionally, for any graph G, χ_t(G) > 0 if and only if G computes PARITY on n inputs.']

Rationale (proposer's reasoning):

> ["Tropical geometry provides a framework to study combinatorial objects using piecewise-linear functions defined over real numbers. This conjecture suggests that the tropical Euler characteristic of a graph's associated matroid can serve as an AC0 parity depth invariant.", 'If true, it would provide a non-trivial connection between tropical geometry and complexity theory by showing how a geometric property of graphs is related to circuit complexity.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c3e2a278f750ecfa`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The tropical Euler characteristic χ_t(G) of graph G is considered supported if it meets the condition χ_t(G) ≥ c·log(n) with a p-value ≤ 0.05, where c is a constant to be determined and n is the number of vertices in G.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "AC0 parity depth" - "graph tropical Euler characteristic" AND "PARITY computation" - "matroid determinant tropicalization" AND AC0 complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if A[j][i] > A[max_row][i]:
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        if pivot == 0:
            continue
        for j in range(i+1, n):
            factor = A[j][i] / pivot
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def tropical_determinant(A):
    n = len(A)
    det = 0
    for sign in itertools.product([1, -1], repeat=n):
        permuted_A = [A[i][:] for i in range(n)]
        for j in range(n):
            if sign[j] == -1:
                for k in range(n):
                    permuted_A[k][j] = -permuted_A[k][j]
        det += sign[0] * gaussian_elimination(permuted_A)[0][0]
    return det
```

```python
def ac0_parity_depth(G):
    n = len(G)
    if n == 1:
        return 1
    if all(G[i][i] == 1 for i in range(n)):
        return 2
    return float('inf')

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(10, 40)
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    for i in range(n):
        G[i][i] = 1
    inc_matrix = []
    for i in range(n):
        row = [G[j][i] for j in range(n)]
        inc_matrix.append(row)
    chi_t = tropical_determinant(inc_matrix)
    depth = ac0_parity_depth(G)
    metric_name = "tropical_euler_characteristic"
    metric_value = chi_t
    instances_tested = 1
    conjecture_holds = chi_t >= math.log(n)
    counterexample = "" if conjecture_holds else f"Graph with n={n} and depth={depth}"
    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30*1000, 100))
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample_desc = next((r["counterexample"] for r in results if not r["conjecture_hold
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, making it impossible to determine if the tropical Euler characteristic χ_t(G) meets the support condition χ_t(G) ≥ c·log(n) with a p-value ≤ 0.05. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11253
2	preregistration	ollama_remote	glm4:latest	0	0	6014
3	novelty	ollama_remote	glm4:latest	0	0	4552
4	novelty	ollama_remote	glm4:latest	0	0	5976
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16949
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11000
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11889
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10574
9	judge	ollama_remote	glm4:latest	0	0	10093

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 88299 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/48da56aad654.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/48da56aad654.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/48da56aad654.tar.gz` (if generated)